

Informatica

Dal file all'informazione

Dario Pescini

Università degli Studi di Milano-Bicocca

Dipartimento di Statistica e Metodi Quantitativi

dario.pescini@unimib.it

Oggi, con il terminale aperto

I lucidi della lezione raccontano come è fatta la macchina. Oggi guardiamo la stessa storia da dove la incontrerete: **un file di dati che qualcun altro ha scritto.**

- Guardare il file **prima** di leggerlo — dieci comandi
- Dai byte ai numeri: separatori, accenti, fine riga
- Perché i numeri mentono: la varianza che diventa negativa
- Quanto pesano i dati, e perché si vettorizza
- Il progetto e la riproducibilità

Tutti gli esempi girano su `MeteoMilano2011.csv` e `2009-2013_iscritti.csv`, che trovate nel materiale del corso. Lo script `esempi_aula.py` rifà ogni numero di questi lucidi.

Com'è fatto, e quanto è grande

```
$ file MeteoMilano2011.csv
MeteoMilano2011.csv: Unicode text, UTF-8 text, with very long lines (450)

$ wc MeteoMilano2011.csv
367   781 31594 MeteoMilano2011.csv
```

- **file** dice la **codifica** e il tipo di fine riga: due cose che decidono se il file si aprirà bene
- **wc** dà righe, parole e byte

367 righe per un anno di misure: **365 giorni + 1 intestazione + 1 riga vuota**. L'aritmetica torna, e se non tornasse sarebbe la prima cosa da capire.

Quali colonne ci sono? una pipeline, un pezzo alla volta

Voglio sapere quali colonne ci sono. Ci arrivo con quattro comandi in fila, e conviene guardarli uno per uno: la **pipe** passa l'uscita di ciascuno all'ingresso del successivo.

```
$ head -2 MeteoMilano2011.csv
```

```
CET, Temperatura maxC, Temperatura mediaC, Temperatura minC, Punto di ru...
```

Due righe, e la prima è **vuota**: è la riga in più che avevamo contato con wc.

```
$ head -2 MeteoMilano2011.csv | tail -1
```

```
CET, Temperatura maxC, Temperatura mediaC, Temperatura minC, Punto di ru...
```

tail -1 tiene l'ultima delle due righe ricevute: la riga vuota sparisce, resta l'intestazione da sola. **Due righe in ingresso, una in uscita**: è il primo stadio che cambia davvero la forma del dato.

...e le colonne diventano righe numerate

```
$ head -2 MeteoMilano2011.csv | tail -1 | tr ',' '\n'
CET
Temperatura maxC
Temperatura mediaC
...
Eventi
WindDirDegrees<br />
```

tr traduce un carattere in un altro: qui ogni virgola diventa un a capo, e la riga unica si apre in **23 righe**, una per colonna.

```
$ head -2 MeteoMilano2011.csv | tail -1 | tr ',' '\n' | nl
1 CET
2 Temperatura maxC
...
22 Eventi
23 WindDirDegrees<br />
```

nl le numera.

🍌 salta fuori il primo guaio: la colonna 23 si chiama `WindDirDegrees<br />`.

Quanti eventi per tipo ci sono stati? Una tabella di frequenza

`cut` taglia una colonna, e per farlo ha bisogno di sapere due cose:

- **il delimitatore dei campi**: `-d ,` (qui la virgola)
- **quale campo tenere**: `-f22` (il numero che ci ha dato `nl`)

```
$ cut -d, -f22 MeteoMilano2011.csv | sort | uniq -c | sort -rn
189
 75 Nebbia
 42 Pioggia
 33 Pioggia-Temporale
 16 Nebbia-Pioggia
  3 Temporale
 ...
  1 Nebbia-Pioggia-Neve
  1 Eventi
```

Poi `sort` mette insieme le righe uguali, `uniq -c` le conta, e un secondo `sort -rn` ordina per conteggio decrescente. La somma fa 367: **torna con `wc`**.

Due cose da notare in quella tabella

Le 189 righe vuote

Non è un difetto: è la prima domanda statistica della giornata. Mancano, o significano «nessun evento»? La risposta cambia ogni analisi che seguirà.

🍌 L'ultima riga: 1 Eventi

cut non sa che esiste un'intestazione: l'ha contata come una categoria. Per escluderla si parte dalla terza riga, `tail -n +3` e questo vale per ogni comando che vedremo.

Una colonna inservibile, trovata in un comando

```
$ tail -n +3 MeteoMilano2011.csv | cut -d, -f19 | grep -c '^$'
341

$ tail -n +3 MeteoMilano2011.csv | cut -d, -f19 | grep -vc '^$'
24
```

La colonna 19 è Max Velocità RafficaKm/h, la raffica massima di vento. Su 365 giorni, **341 sono vuoti e solo 24 hanno un valore**: manca il **93%** delle misure.

`grep -c '^$'` conta le righe vuote — ^ inizio riga, \$ fine riga, niente in mezzo — e `grep -vc` conta quelle che **non** lo sono. I due numeri sommano a 365: il conto torna.

Con **pandas** lo scoprirete comunque — ma dopo aver caricato il file, scritto il codice e magari fatto un grafico. Qui è costato un comando, prima di tutto il resto.

🔊 **sapere cosa c'è in un file è un lavoro che si fa prima di aprirlo.**

Ci sono giorni ripetuti? E qual è stato il più caldo?

```
$ tail -n +3 MeteoMilano2011.csv | cut -d, -f1 | sort | uniq -d | wc -l
0
# nessuna data duplicata: 365 giorni distinti

$ tail -n +3 MeteoMilano2011.csv | sort -t, -k2 -rn | head -3 | cut -d, -f1,2
2011-9-13,30
2011-6-27,29
2011-8-31,29
```

- `uniq -d` mostra solo i **doppioni**: zero righe, nessuna data ripetuta
- `sort -t, -k2 -rn` ordina sul secondo campo, numericamente, al contrario: i tre giorni più caldi del 2011

Un conteggio che non torna, e perché

```
$ grep -c Pioggia MeteoMilano2011.csv  
98
```

Nella tabella di frequenza «Pioggia» da sola era **42**. Chi ha ragione?

Entrambi: `grep -c` conta le righe che contengono la parola, e quindi somma tutte le categorie composte — Pioggia, Pioggia-Temporale, Nebbia-Pioggia, Pioggia-Neve, Pioggia-Grandine-Temporale, Nebbia-Pioggia-Neve. Sono 98.

🔊 un conteggio senza la definizione di cosa si conta non è un dato. È lo stesso errore che si fa con un `value_counts()` su categorie composte.

Il file è quello che credi?

```
$ ls -lh MeteoMilano2011.csv
-rw-r--r--  1 dario  staff   31K 20 Sep 14:12 MeteoMilano2011.csv

$ shasum -a 256 MeteoMilano2011.csv
a6855408488c81a1ae3d6ee7... MeteoMilano2011.csv

$ du -sh data/
36K      data/
```

- data e dimensione rispondono a «*è il file che ho scaricato martedì?*»
- l'impronta **shasum** risponde a «*è lo stesso file, bit per bit?*» — due download della stessa pagina non sempre lo sono

Dieci comandi, dieci domande

Comando	La domanda a cui risponde
file	com'è codificato, che fine riga ha
wc	quante righe, quanti byte
head, tail	come sono fatte le prime righe
head tr nl	quali colonne, con che numero
cut	estrai una colonna
sort uniq -c	tabella di frequenza
uniq -d	ci sono doppioni?
grep -c	quante righe contengono ...
sort -k	ordina su una colonna
ls -lh, shasum	è il file giusto, è cambiato?

Sono in `L02_riepilogo.md`, e gli esercizi li fanno usare uno per uno.

Quando pandas tace

```
>>> df = pd.read_csv("MeteoMilano2011.csv")
>>> df.shape
(365, 23)
>>> df.columns[-1]
'WindDirDegrees<br />'
>>> df.iloc[:3, -1].tolist()
['225<br />', '10<br />', '100<br />']
```

Nessun errore, nessun avviso: 365 righe e 23 colonne, tutto normale. Ma la direzione del vento è **testo**, non gradi — e resterà testo fino al primo grafico che non funziona.

Il `<br />` è spazzatura HTML rimasta dalla pagina da cui il file è stato salvato. `head` l'avrebbe mostrata subito.

Quando pandas urla

```
>>> pd.read_csv("2009-2013_iscritti.csv")
ParserError: Error tokenizing data. C error:
Expected 1 fields in line 288, saw 2

>>> pd.read_csv("2009-2013_iscritti.csv", sep=";").shape
(954, 9)
```

Il file MIUR degli iscritti per ateneo usa il **punto e virgola**, come quasi tutti i dati pubblici italiani: senza `sep= ' ; '` pandas vede una sola colonna, e alla riga 288 trova una virgola dentro un nome e si arrende.

Dei due comportamenti, **il primo è più pericoloso del secondo**: l'errore ti fermi, il silenzio no.

Lo zero davanti

```
>>> pd.read_csv("2009-2013_iscritti.csv", sep=";")["COD_ATENEO"].head(3).tolist()
[101, 102, 201]

>>> pd.read_csv("2009-2013_iscritti.csv", sep=";",
...             dtype={"COD_ATENEO": str})["COD_ATENEO"].head(3).tolist()
['00101', '00102', '00201']
```

Il codice dell'Università di Torino è 00101. Letto come numero diventa 101, e da quel momento non è più un codice: non si può più unire con un'altra tabella, e stampato in un report è sbagliato.

Vale per **CAP**, **codice fiscale**, **codici comune ISTAT**: sequenze di cifre che non sono numeri. La cura è dichiararlo — `dtype=str`.

I byte di una lettera accentata

```
$ head -2 MeteoMilano2011.csv | tail -1 | cut -d, -f8 | xxd
00000000: 4d61 7820 556d 6964 6974 c3a0          Max Umidit..
```

Dodici byte per undici caratteri: la à finale ne occupa **due**, c3 a0. I 128 caratteri dei 7 bit ASCII non bastavano per gli alfabeti del mondo, e UTF-8 li estende usando più byte per i caratteri meno comuni.

```
>>> campo.decode("utf-8")      'Max Umidità'
>>> campo.decode("latin-1")    'Max UmiditÃ'
```

Gli stessi byte, letti con la convenzione sbagliata. È tutto qui il mistero dei Città e dei perché nei file scaricati: nessun dato è corrotto, è la **codifica dichiarata** che è sbagliata — in pandas, `encoding=`.

Unix e Windows: il fine riga che spacca i conti

```
$ file meteo_windows.csv
meteo_windows.csv: UTF-8 text, with CRLF line terminators

$ tail -n +2 meteo_windows.csv | cut -d, -f23 | cat -A | head -2
225<br />^M$
10<br />^M$
```

Su Unix la riga finisce con un byte, `0a`; su Windows con due, `0d 0a`. Il `0d` in più — che `cat -A` mostra come `^M` — finisce **dentro l'ultimo campo**.

Conseguenza: in una tabella di frequenza `Nebbia` e `Nebbia^M` diventano **due categorie diverse**. La differenza fra i due file system di cui parlano i lucidi si presenta così, nel conteggio che non torna.

Il tipo float non è l'insieme dei reali

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
>>> sum([0.1] * 10)
1.0
```

Un numero con la virgola sta in 64 bit, e 0.1 in binario è periodico: si memorizza il più vicino possibile. Da lì il primo risultato.

Il terzo dice l'altra metà: dieci errori si sono compensati, e la somma è esatta. **Non è una regola, è un caso** — e per questo due formule diverse dello stesso conto possono dare risposte diverse.

La varianza, in due modi

La pressione a Milano nel 2011: **365 misure, media 1017.11 hPa**. Due formule equivalenti a scuola:

ingenua: $\overline{X^2} - \bar{X}^2$

due passate: $\overline{(X - \bar{X})^2}$

	ingenua	due passate
float64	38.552374	38.552374
float32	38.437500	38.552380

In doppia precisione le due coincidono. In singola, la prima sbaglia già la terza cifra: $\overline{X^2}$ e $\bar{X}^2$ sono due numeri grandi e quasi uguali, e sottraendoli le cifre buone si cancellano.

La stessa varianza, con un offset

Stessi dati, più 10^8 – un sensore mal calibrato, o un identificativo numerico usato come misura:

	ingenua	due passate
float64	38.000000	38.552374
float32	-1073741824.000000	92.230141

Una varianza negativa

Non è un bug di Python: è il risultato di aver chiesto la differenza fra due numeri enormi e quasi identici. Una quantità che **per definizione** non può essere negativa lo diventa, e nessun messaggio d'errore avvisa.

Cosa portarsi via

- due formule **algebricamente** equivalenti non sono **numericamente** equivalenti
- è la ragione per cui **np.var** e pandas calcolano la varianza in due passate, e non con la formula che sta sui libri
- quando un risultato ha un ordine di grandezza inatteso, prima di cercare l'errore nei dati vale la pena guardare i **tipi**

E la **regola pratica**: centrare i dati prima di calcolare momenti. Sottrarre la media costa un passaggio e salva tutte le cifre.

Su disco e in memoria

```
$ ls -lh MeteoMilano2011.csv          # 31K

>>> df.memory_usage(deep=True).sum() / 1024
74.2
```

Trentuno kilobyte di testo diventano settantaquattro in memoria: il file è una sequenza di caratteri, il **DataFrame** è una tabella di numeri tipizzati più i suoi indici.

Su un file da 31 KB non interessa a nessuno. Su un file da 2 GB il fattore due decide se l'analisi parte o se il portatile si blocca — e pandas non avvisa prima, ci prova.

Il costo di una colonna

Una colonna da **dieci milioni** di righe:

tipo	memoria	quando basta
float64	76 MB	misure con decimali, il default
float32	38 MB	misure con 7 cifre significative
int16	19 MB	conteggi fino a 32767
int8	10 MB	codici, flag, valori 0-127

Venti colonne così sono **1,5 GB** oppure **200 MB**: la differenza la decidete voi con `dtype=` al momento della lettura, insieme a `usecols=` per non leggere le colonne che non servono.

Perché esiste il codice vettorializzato

```
# un milione di righe, la stessa operazione
ciclo Python  for v in s: tot += v*2      0.032 s
vettorizzato  (s * 2).sum()              0.001 s    -> ~40x
```

I lucidi hanno mostrato il ciclo della CPU: prendi l'istruzione, decodificala, esegui. Il processore ne fa una alla volta, molto in fretta.

- il **ciclo Python** fa un milione di giri dell'**interprete**: per ogni riga controlla i tipi, crea oggetti, chiama funzioni
- l'**operazione vettorializzata** fa un giro e passa il lavoro a codice compilato, che scorre la memoria di fila

Non è che «pandas è più veloce»: è che il lavoro scende al livello di cui parlano i lucidi, invece di restare nell'interprete.

La forma di un'analisi

```
analisi_meteo/  
  data/raw/      i file come li ho scaricati      chmod 444  
  data/clean/    quelli che produco io  
  script/        il codice  
  out/           grafici e tabelle: si rigenerano
```

- i **grezzi non si toccano**: `chmod 444` li rende non scrivibili, e se un comando prova a sovrascriverli fallisce invece di riuscire
- tutto ciò che sta in `out/` deve poter essere buttato e rifatto lanciando il codice: se non è così, quel risultato non è riproducibile
- i percorsi **relativi** rendono la cartella trasportabile — ed è il motivo per cui `pd.read_csv("data/raw/meteo.csv")` dipende da `pwd`

I nomi dei file sono dati

Da evitare

- Dati finali (2).csv
- analisi_def_CORRETTA.py
- meteo 12-3-11.csv

Da preferire

- 2011_meteo_milano.csv
- lettura_meteo.py
- 2026-09-20_estrazione.csv

- niente spazi e niente accenti: sulla riga di comando gli spazi separano gli argomenti, e gli accenti cambiano da una macchina all'altra
- date in formato **ISO** aaaa-mm-gg: l'ordine alfabetico diventa l'ordine cronologico, gratis
- maiuscole e minuscole contano su Linux: Dati.csv e dati.csv sono due file

L'ambiente fa parte del risultato

```
pandas 3.0.5 . numpy 2.5.2 . python 3.14.7  
COD_ATENEO qui e ora: dtype int64
```

Preparando questi lucidi, lo stesso file letto con lo stesso comando ha dato
una volta un tipo intero e una volta una stringa, su due versioni diverse di pandas.

«Funziona sul mio computer»

Un'analisi non è solo il codice: è il codice **più** le versioni delle librerie che lo eseguono. Per questo si lavora in un **ambiente** dichiarato — sulla VM del corso lo abbiamo tutti uguale — e per questo la prima riga di uno script utile stampa le versioni che sta usando.

Cosa vi porto via da oggi

- 1 guarda il file prima di leggerlo: dieci comandi, trenta secondi, e sai già cosa manca e cosa non torna
- 2 un CSV è testo: separatore, codifica, fine riga sono decisioni di chi l'ha scritto, e vanno scoperte, non indovinate
- 3 i tipi contano: uno zero davanti, un float32, una colonna letta come stringa cambiano il risultato senza un messaggio d'errore
- 4 i dati occupano memoria e il codice vettorializzato esiste per come è fatta la macchina
- 5 la riproducibilità è un'abitudine: grezzi in sola lettura, nomi ordinabili, ambiente dichiarato

Da leggere: L02_shell.md e L02_riepilogo.md. Da provare: L02_esercizi.md. Da rifare: python esempi_aula.py.